

Optimized Kernel Fusion for Accelerating Multimodal Inference on Edge Device

트랙: 연구

팀명: Nitro

팀원: 2217011 박세진, 2271029 박윤형, 2376004 강선화

지도교수: 윤명국

Table of contents

I. Team Info.

- A. Project Title
- B. Team Information
- C. Team Members

II. Project Summary

- A. Problem Definition
- B. Comparison with Existing Research
- C. Proposed Methodology
- D. Expected Effects and Significance
- E. Key Features

III. Project Design

- A. Requirements Definition
- B. Overall System Architecture

I. Team Information

A. Project Title

Optimized Kernel Fusion for Accelerating Multimodal Inference on Edge Device

B. Team Information

29번 팀 Nitro

C. Team Members

- 강선화(2376004):
 - Multimodal Model 구조 분석 및 프로파일링
 - Kernel Fusion 구현
 - 성능 비교 실험 진행
 - 문서 정리 및 회의록 관리
- 박세진(2217011):
 - 프로젝트 전체 연구 방향 설계
 - Multimodal Model 구조 분석 및 프로파일링
 - Kernel Fusion 구현
 - 성능 비교 실험 진행
 - 문서 정리 및 회의록 관리
- 박윤형(2271029):
 - Multimodal Model 구조 분석 및 프로파일링
 - Kernel Fusion 구현
 - 성능 비교 실험 진행
 - 문서 정리 및 회의록 관리

II. Project Summary

A. Project Definition

최근 멀티모달 대형 모델(Large Multimodal Models, LMMs)은 텍스트와 이미지 등 다양한 모달리티를 통합적으로 처리하기 위해 Transformer 기반 구조를 채택하고 있으며, 성능 향상을 위해 모델 규모, 입력 해상도, 그리고 visual token 수가 지속적으로 증가하고 있다. 이러한 변화는 연산량과 메모리 요구량을 크게 증가시키며, 특히 attention 및 feed-forward network(FFN)에서 수행되는 대규모 행렬 연산(GEMM)의 비중을 확대시키고 있다.

Attention 메커니즘은 Query, Key, Value(QKV) 간의 유사도를 기반으로 입력 요소 간의 중요도를 계산하는 연산으로, 대부분 matrix multiplication과 softmax 연산으로 구성된다. 이로 인해 Transformer 기반 모델에서는 GEMM과 attention 연산이 전체 계산의 대부분을 차지한다.

그러나 엣지 디바이스 환경에서는 이러한 연산 구조가 비효율적으로 실행된다. Embedded GPU는 제한된 연산 자원과 메모리 대역폭을 가지며, 여러 kernel을 완전히 병렬적으로 실행하는 데 제약이 존재한다. 또한 multistream execution 환경에서도 kernel 간 완전한 병렬성이 보장되지 않아 context switching overhead가 발생하며, 이는 전체 latency 증가로 이어진다.

Transformer block 내부에서는 GEMM과 같은 대규모 연산뿐 아니라, QKV projection 이후의 split, reshape, transpose, 그리고 bias addition, activation, layer normalization과 같은 연산이 각각 독립적인 CUDA kernel로 실행된다. 이로 인해 하나의 Transformer block 실행 과정에서 수십 개 이상의 kernel이 순차적으로 호출되며, kernel launch overhead와 global memory access 비용이 누적된다.

특히 QKV split 및 transpose 연산은 계산량은 거의 없고 메모리 접근이 대부분을 차지하는 memory-bound kernel로, 전체 attention pipeline에서 의미 있는 latency를 차지하는 병목으로 나타난다. 이러한 특성은 Nsight Systems 및 Nsight Compute의 roofline 분석을 통해 확인되며, 연산 대비 메모리 접근 비율이 높은 비효율적인 실행 패턴을 보인다.

따라서 본 연구에서는 이러한 병목을 해결하기 위해 Transformer 내부 연산의 kernel 실행 구조를 재구성하고, kernel launch overhead 및 memory traffic을 동시에 감소시키는 접근이 필요하다고 판단하였다.

B. Comparison with Existing Research

GPU inference 최적화를 위한 대표적인 방법으로는 TensorRT와 CUTLASS 기반 kernel 최적화 기법이 존재한다.

TensorRT는 연산 그래프 수준에서 사전에 정의된 operator pattern을 기반으로 kernel fusion을 수행하는 rule-based heuristic 접근을 사용한다. 이 방식은 CNN 및 표준 Transformer workload에 대해서는 효과적이지만, attention preparation, QKV transformation, epilogue computation과 같은 비정형적인 연산 흐름에서는 fusion 기회를 충분히 활용하지 못하는 한계를 가진다. 특히 일부 Transformer 연산은 여전히 다수의 kernel로 분리되어 실행되어 kernel launch overhead를 유발한다.

CUTLASS는 텐서 코어 기반의 고성능 GEMM 연산을 제공하며, Efficient GEMM Template(EVT)를 통해 epilogue 단계에서 element-wise 연산을 fusion할 수 있다. 그러나 이 접근은 activation, bias addition 등 단순 연산에 주로 적용되며, QKV split, reshape, transpose와 같은 memory-bound tensor transformation을 포함한 전체 연산 흐름까지 통합하기에는 제한적이다. 또한 kernel configuration 변경 시 재컴파일이 필요하며, 개발 복잡도가 증가하는 문제도 존재한다.

결과적으로 기존 방법들은 Transformer block 전반에서 발생하는 kernel launch overhead 및 memory-bound 연산을 구조적으로 줄이기에는 한계가 있으며, 특히 엣지 디바이스 환경에서의 비효율을 충분히 해결하지 못한다.

C. Proposed Methodology

본 연구에서는 멀티모달 모델의 Transformer block에서 발생하는 성능 병목을 정량적으로 분석하고, 이를 기반으로 kernel-level 최적화를 수행하는 접근을 제안한다.

먼저, Nsight Systems를 활용하여 Transformer block 내부에서 실행되는 CUDA kernel들의 호출 패턴을 분석하였다. 분석 결과, 하나의 Transformer block 실행 과정에서 GEMM과 같은 대규모 연산뿐 아니라, QKV projection 이후의 split, transpose, reshape, 그리고 bias addition, activation, layer normalization과 같은 다수의 연산이 각각 독립적인 kernel로 수행되며, 수십 개 이상의 kernel이 순차적으로 호출되는 구조를 확인하였다. 이러한 구조는 kernel launch overhead의 누적과 함께 global memory access 증가로 이어져 전체 latency 증가의 주요 원인으로 작용한다.

이후 Nsight Compute를 이용한 roofline 분석을 통해 각 kernel의 성능 특성을 평가하였다. 그 결과, QKV split 및 transpose 연산은 낮은 arithmetic intensity를 가지며 memory bandwidth에 의해 성능이 제한되는 대표적인 memory-bound kernel로 나타났다. 또한 해당 연산은 계산량은 작지만 전체 attention pipeline에서 의미 있는 latency 비중을 차지하는 것으로 확인되었다. 이러한 분석을 바탕으로, 본 연구에서는 주요 병목 구간을 다음 두 가지로 정의하였다.

1) QKV Split + Transpose Fusion

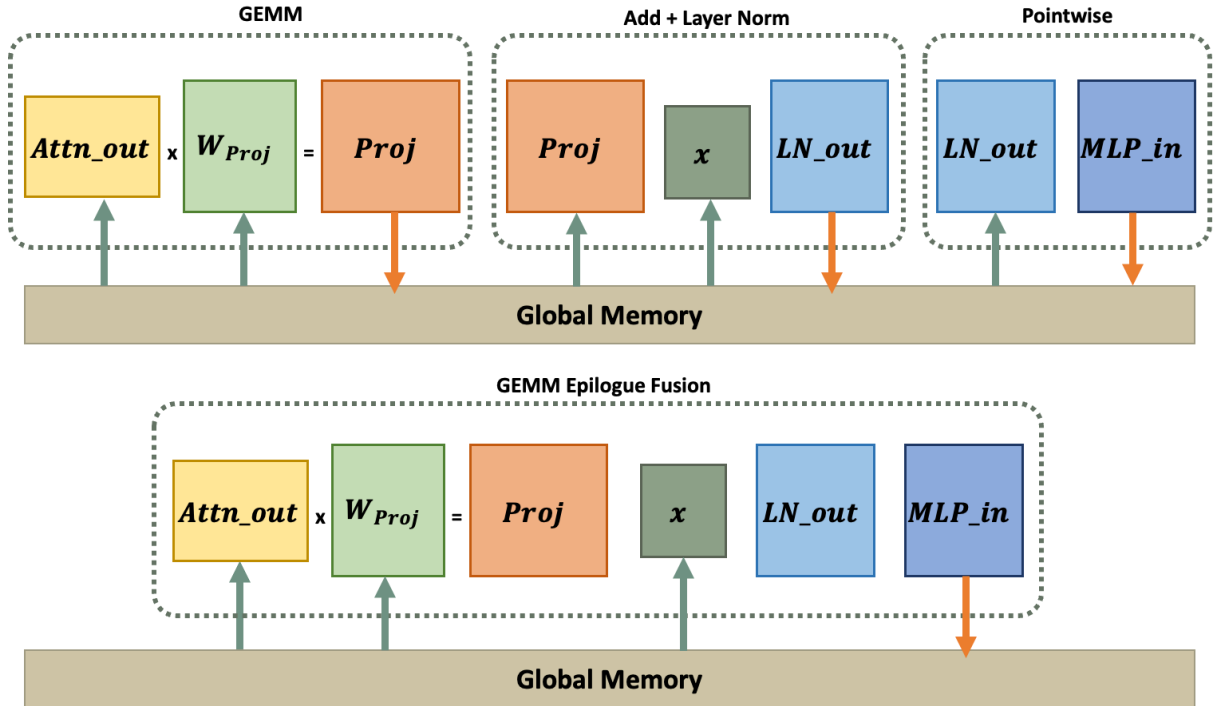


그림 1. QKV Split & Transpose 커널 퓨전.

QKV split 및 transpose 연산은 연산량 대비 메모리 접근 비중이 높은 memory-bound kernel로, 반복적인 global memory read/write를 유발하며 latency에 비효율적으로 기여한다. Profiling 결과, 해당 kernel은 전체 attention 연산 과정에서 무시할 수 없는 latency 비중을 차지하며, 특히 엡지 디바이스 환경에서 그 영향이 더욱 크게 나타난다.

이를 해결하기 위해, 본 연구에서는 QKV projection GEMM의 output 단계에서 split 및 transpose 연산을 동시에 수행하도록 kernel을 재구성하는 fusion 전략을 제안한다. 이를 통해 기존에 별도의 kernel로 수행되던 tensor transformation 연산을 제거하고, intermediate tensor의 global memory 접근을 최소화한다.

이러한 접근은 memory-bound kernel의 실행을 제거함으로써 memory traffic을 감소시키고, kernel launch 횟수를 줄여 전체 latency 개선으로 이어질 것으로 기대된다. 또한 해당 기법은 서버 GPU뿐 아니라 엡지 디바이스 환경에서도 적용 가능성을 검증하기 위해 다양한 환경에서 성능을 측정할 예정이다.

2) GEMM Epilogue Fusion (Residual + Bias + Activation + LayerNorm)

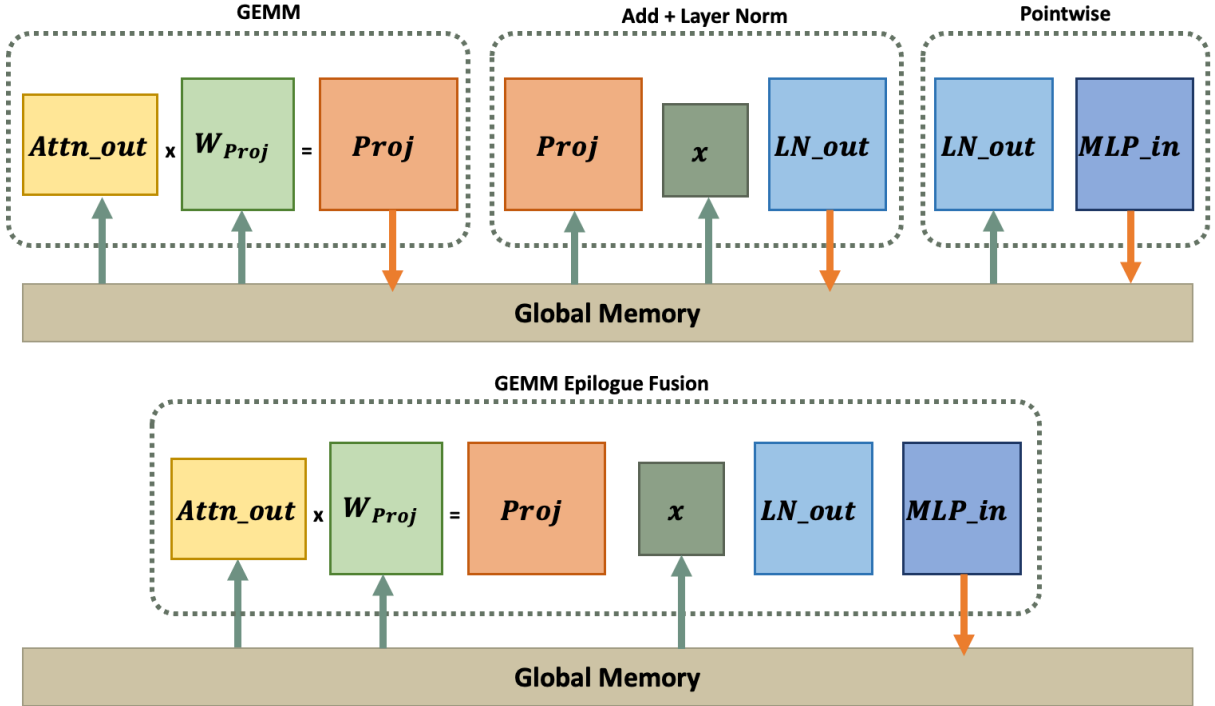


그림 2. GEMM 에필로그 퓨전.

Transformer block에서는 GEMM 이후 residual addition, bias addition, activation, layer normalization과 같은 연산이 연속적으로 수행되며, 각각이 별도의 kernel로 실행된다. 이러한 구조는 intermediate tensor를 global memory에 반복적으로 저장하고 다시 읽어오는 과정을 발생시켜 memory access overhead를 증가시킨다. 본 연구에서는 이러한 연산들을 GEMM kernel의 epilogue 단계에 통합하여, 하나의 fused kernel에서 연속적으로 수행하는 방식을 제안한다. 이를 통해 kernel 호출 횟수를 감소시키고, intermediate tensor의 불필요한 memory access를 제거할 수 있다.

특히 layer normalization은 reduction 연산을 포함하기 때문에 단순한 element-wise 연산과 달리 thread 간 synchronization과 tile-based parallelization이 요구되며, 이에 따른 kernel 설계의 복잡성이 존재한다. 따라서 해당 fusion은 단순 연산 결합을 넘어, 효율적인 병렬 실행 구조를 고려한 커널 설계가 필요하다.

현재 GEMM epilogue fusion은 구현 및 최적화가 진행 중이며, 향후 profiling 기반 성능 분석을 통해 그 효과를 검증할 예정이다.

종합적으로, 본 연구는 Nsight 기반 profiling을 통해 Transformer block 내부의 kernel execution 패턴과 memory-bound 병목을 식별하고, 이를 해결하기 위한 QKV fusion과 GEMM epilogue fusion의 두 가지 kernel-level 최적화 전략을 제안한다. 제안된 kernel fusion은 CUTLASS 기반 GEMM 구조 및 epilogue 확장 메커니즘을 활용하여 구현되며, 기존 연산 흐름을 유지하면서 kernel-level에서의 연산 통합을

수행한다. 이러한 접근을 통해 kernel launch overhead와 memory traffic을 동시에 감소시키고, 멀티모달 모델의 inference latency를 개선하는 것을 목표로 한다.

D. Expected Effects and Significance

본 연구를 통해 다음과 같은 효과를 기대할 수 있다.

- kernel fusion을 통해 kernel launch 횟수를 감소시켜 launch overhead를 줄일 수 있다.
- memory-bound kernel을 제거하고 intermediate tensor의 global memory 접근을 최소화하여 memory traffic을 감소시킬 수 있다.
- Transformer block 내부 연산의 실행 효율을 개선하여 전체 inference latency를 단축할 수 있다.
- 엣지 디바이스 환경에서 멀티모달 모델의 실시간 추론 가능성을 향상시킬 수 있다.

또한 본 연구는 Transformer 기반 연산 패턴을 중심으로 최적화를 수행함으로써, CLIP, BLIP-2, SigLIP-2, Qwen2.5-VL과 같이 서로 다른 구조의 멀티모달 모델에 공통적으로 적용 가능한 범용적 최적화 기법을 제안한다.

현재 QKV fusion에 대해서는 profiling 기반 분석과 구현을 통해 성능 개선 가능성을 확인하였으며, GEMM epilogue fusion은 구현 및 최적화가 진행 중이다.

E. Key Features

본 연구에서 구현 및 검증할 주요 기능은 다음과 같다.

- Nsight Systems를 활용한 Transformer block 내부 kernel execution 분석
- Nsight Compute roofline 기반 memory-bound kernel 식별
- QKV split 및 transpose kernel의 latency 비중 분석 및 최적화
- QKV fusion kernel 설계 및 구현, 성능 평가 (서버 및 엣지 환경 포함)
- GEMM epilogue fusion kernel 설계 및 구현 (residual, bias, activation, layer normalization 포함)
- TensorRT baseline과의 성능 비교
- 다양한 멀티모달 모델(CLIP, BLIP-2, SigLIP-2, Qwen2.5-VL)에 대한 적용 및 검증
- 성능 평가 지표 분석
 - inference latency
 - kernel launch overhead
 - memory traffic
 - accuracy 변화 여부

III. Project-Design

A. Requirements Definition

본 과제의 핵심 요구사항은 멀티모달 모델의 Transformer block 내부에서 발생하는 비효율적인 kernel execution 구조를 정량적으로 분석하고, 이를 개선하기 위한 kernel fusion 기법을 설계 및 검증하는 것이다. 최근 Transformer 기반 멀티모달 모델은 attention 및 FFN 중심의 대규모 연산 구조를 가지며, 특히 block 내부에서 GEMM 이후에 다수의 tensor transformation 및 element-wise 연산이 독립적인 CUDA kernel로 분리되어 수행된다. 이러한 구조는 kernel launch overhead를 증가시키고 intermediate tensor의 반복적인 global memory access를 유발하므로, 전체 추론 지연시간 증가의 주요 원인으로 작용한다. 따라서 본 연구에서는 Nsight 기반 profiling을 통해 병목 kernel을 식별하고, 이를 바탕으로 kernel-level fusion을 수행하여 memory traffic과 launch overhead를 동시에 감소시키는 것을 주요 요구사항으로 설정한다.

우선 Transformer block 내부의 kernel 호출 구조를 세밀하게 분석할 필요가 있다. Nsight Systems 분석 결과, Transformer block은 단일 대규모 연산으로 구성되지 않으며, `CLS token concat + position embedding add + layer normalization`, `QKV bias add/reshape`, `QKV projection`, `Q, K, V split + transpose`, `multi-head attention`, `attention output projection`, `residual add + layer normalization`, `MLP bias add/reshape`, `MLP FC1`, `output initialization`, `MLP FC2`와 같은 다수의 kernel이 순차적으로 호출되는 구조를 가진다. 즉, Tensor Core 기반 GEMM kernel뿐 아니라 reshape, transpose, add, normalization과 같은 보조 연산들이 block 내부에 빈번하게 삽입되어 있으며, 이들이 각각 독립 kernel로 수행됨으로써 연산 흐름이 과도하게 세분화되어 있다. 따라서 fusion 대상 연산을 도출하기 위해서는 Transformer block 내 kernel sequence를 분해하고, 각 kernel의 기능과 실행 시간 비중을 함께 파악해야 한다.

이와 함께, 연산 집약도는 낮고 메모리 접근 비중은 높은 memory-bound kernel을 식별하는 과정이 필요하다. profiling 결과, Transformer block 내의 모든 kernel이 동일한 병목 특성을 보이는 것은 아니며, 특히 `QKV bias add/reshape`, `Q, K, V split + transpose`, `Residual Add + LayerNorm`, `MLP bias add/reshape`와 같은 연산들은 높은 FLOPs를 요구하지 않음에도 불구하고 대량의 tensor read/write를 수반한다. 이러한 kernel들은 arithmetic intensity가 낮고, roofline 관점에서 memory bandwidth에 의해 성능이 제한되는 전형적인 memory-bound workload에 해당한다. 그중에서도 `Q, K, V split + transpose`는 attention preparation 단계에서 반복적으로 수행되며, intermediate tensor를 global memory에 저장한 후 다시 읽어오는 과정을 수반하므로 execution time과 DRAM access 측면에서 모두 비효율적인 병목으로 작용한다. 또한 residual add, bias add, layer normalization 역시 element-wise 연산과 reduction이 결합된 구조로 인해 연산 밀도는 낮은 반면 memory access 비중은 높다. 따라서 본 연구에서는 단순히 실행 시간이 긴 kernel만을 대상으로 삼는 것이 아니라, 실제 성능 저하를 유발하는 low-AI memory-bound kernel을 우선적인 최적화 대상으로 정의한다.

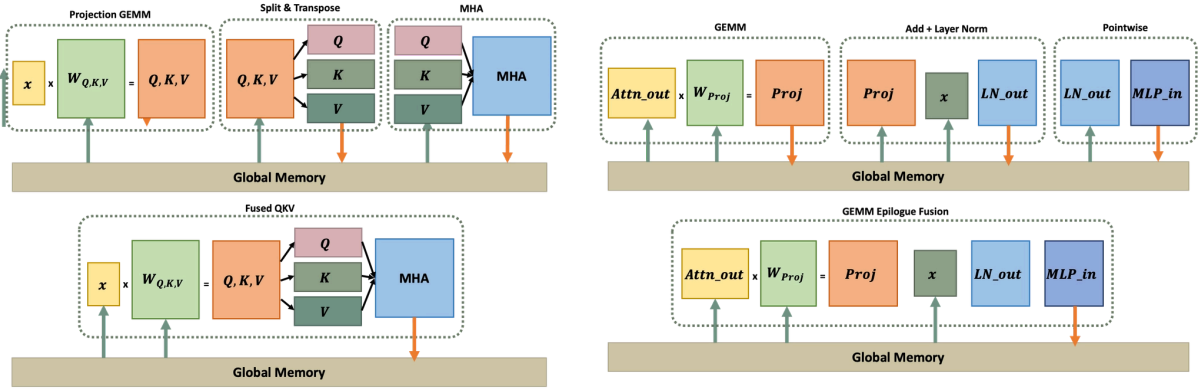


그림 3. (1) QKV Fusion, (2) GEMM+Residual Add+LN+Bias Add Fusion

이러한 분석을 바탕으로, 본 연구에서는 두 가지 kernel fusion 전략을 핵심 solution으로 설정한다. 첫 번째는 QKV Fusion으로, QKV projection GEMM 이후 별도의 kernel로 수행되던 split 및 transpose를 GEMM의 output 단계에 통합하는 방식이다. 기존의 `GEMM → global memory write → split/transpose kernel → global memory write → MHA input` 구조를 `GEMM + split/transpose fused kernel → MHA input` 구조로 재구성함으로써, intermediate tensor 저장 및 재로드를 최소화할 수 있다. 두 번째는 GEMM + Residual Add + LayerNorm + Bias Add Fusion으로, attention output projection 또는 MLP projection 이후 수행되던 residual addition, bias addition, layer normalization을 GEMM epilogue에 통합하는 방식이다. 이를 통해 intermediate tensor를 global memory에 반복적으로 기록하지 않고 연속적으로 처리할 수 있으므로, kernel launch 횟수와 DRAM traffic을 동시에 줄일 수 있다. 특히 layer normalization은 reduction 및 synchronization이 수반되므로, 단순한 element-wise fusion보다 높은 구현 복잡도를 가지며, tile-based parallelization과 numerically stable reduction을 고려한 설계가 요구된다.

또한 제안한 각 feature의 효과를 검증하기 위한 실험 체계 역시 요구사항에 포함된다. QKV Fusion의 경우 TensorRT baseline 대비 fused implementation의 효과를 정량적으로 분석하기 위해, 모델별 sequence length 변화에 따른 `GEMM 대비 split_transpose의 실행 시간 비율`, `GEMM 대비 split_transpose의 DRAM access 비율`, 그리고 최종 `speedup`을 측정해야 한다. 실험 대상은 encoder 계열과 decoder 계열을 모두 포함하도록 CLIP-base-text, BERT-base, BERT-large, OPT-2.7B, Llama-7B로 구성하며, sequence length 증가에 따라 split_transpose의 병목 정도가 어떻게 변화하는지를 확인한다. GEMM Epilogue Fusion에 대해서는 구현 완료 후 baseline과 비교하여 kernel launch count, DRAM transaction 수, arithmetic intensity, block latency, 그리고 정확도 변화를 함께 평가해야 한다. 본 연구의 목적은 단순한 속도 향상에 그치지 않고 실제 멀티모달 모델 적용 가능성을 입증하는 데 있으므로, fused kernel 적용 이후에도 output correctness가 유지되는지 반드시 확인해야 한다.

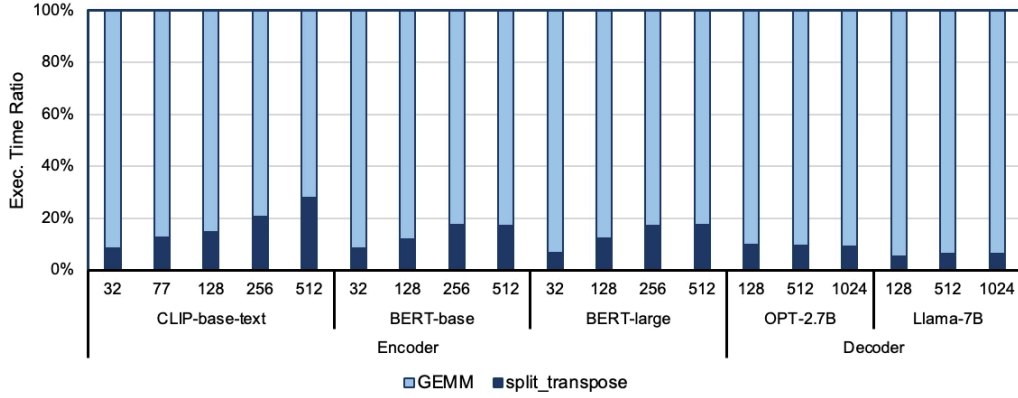


그림 4. GEMM & Split_Transpose Kernel Exec Time Ratio

QKV Fusion의 구현 결과는 이러한 요구사항의 타당성을 뒷받침한다. 먼저 execution time ratio 분석 결과, `split_transpose` kernel은 모든 모델에서 무시하기 어려운 실행 시간 비중을 차지하였으며, sequence length가 증가할수록 그 비중 또한 증가하는 경향을 보였다. 특히 encoder 계열에서는 긴 sequence에서 `split_transpose`의 비중이 더욱 뚜렷하게 증가하였고, CLIP-base-text의 경우 sequence length 512에서 약 30%에 근접하는 수준까지 상승하였다. 이는 split/transpose가 단순한 보조 연산이 아니라, 긴 입력 조건에서 attention preparation 단계의 실질적인 병목이 될 수 있음을 의미한다. 반면 decoder 계열에서는 GEMM의 계산 비중이 상대적으로 크게 유지되었으나, 그림에도 `split_transpose`가 일정 수준의 latency overhead를 지속적으로 유발하는 것으로 나타났다.

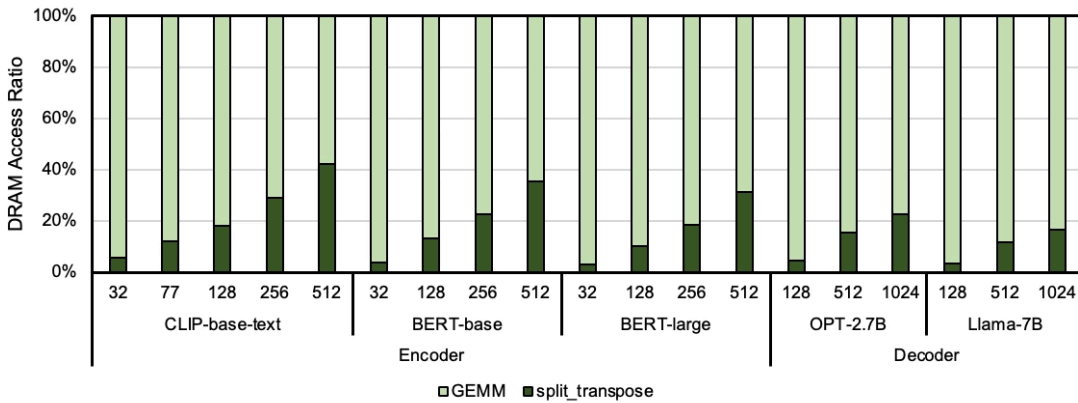


그림 5. GEMM & Split_Transpose DRAM Access Ratio

DRAM access ratio 분석에서는 이러한 경향이 더욱 분명하게 드러났다. `split_transpose`는 execution time 비중 이상으로 높은 DRAM access 비중을 보였으며, 일부 설정에서는 전체 DRAM access의 30~40% 수준을 차지하였다. 이는 해당 kernel이 연산량 자체는 크지 않음에도 tensor layout 변환 과정에서 과도한 memory traffic을 유발하는 전형적인 memory-bound kernel임을 보여준다. 특히 sequence length가 길어질수록 이 비중이 빠르게 증가하였는데, 이는 split 및 transpose 연산이 input token 수 증가에 비례하여 memory movement를 확대시키기 때문이다. 따라서 이 kernel을 제거하거나 GEMM과

통합하는 방식은 단순한 kernel 개수 축소를 넘어, attention stage 전체의 메모리 병목을 구조적으로 완화하는 효과를 기대할 수 있다.

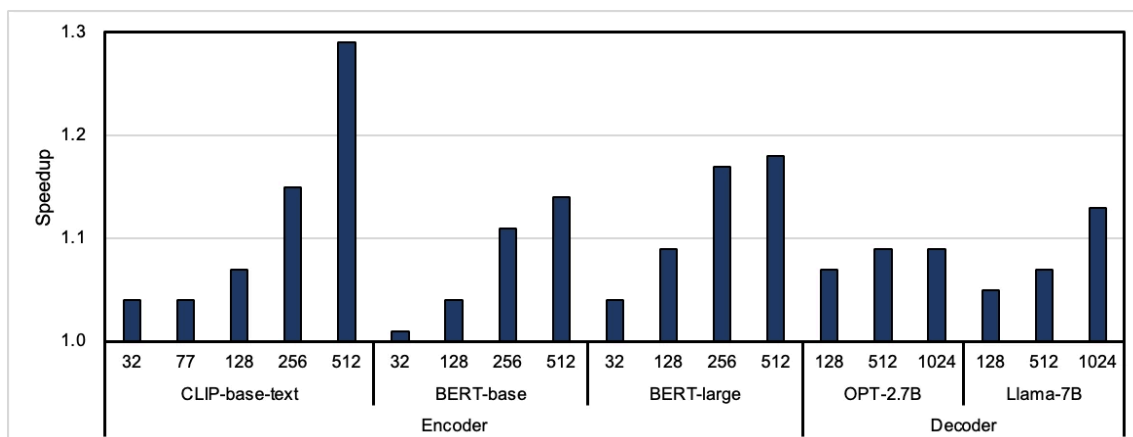


그림 6. 최종 Speedup

최종 speedup 결과 역시 앞선 분석과 일관된 경향을 나타낸다. QKV Fusion 적용 시 모든 모델에서 baseline 대비 1.0 이상의 성능 향상이 관찰되었으며, 특히 sequence length가 증가할수록 speedup이 커지는 경향이 뚜렷하게 확인되었다. CLIP-base-text의 경우 긴 sequence에서 최대 약 1.29배의 성능 향상이 관찰되었고, BERT-base 및 BERT-large에서도 1.1배 이상의 개선이 확인되었다. OPT-2.7B와 Llama-7B와 같은 decoder 계열에서는 개선 폭이 상대적으로 완만하였으나, 일관된 speedup이 유지되었다. 이는 split_transpose의 execution time 및 DRAM access 비중이 큰 조건일수록 fusion의 효과가 직접적으로 확대됨을 의미한다. 반대로 짧은 sequence에서는 split_transpose 자체의 절대 비용이 작기 때문에 speedup이 제한적으로 나타났는데, 이는 해당 구간에서는 fixed launch overhead와 GEMM 중심의 계산 비용이 전체 latency에서 차지하는 비율이 더 크기 때문이다. 또한 hidden dimension이 큰 decoder 계열에서는 GEMM의 계산 비중이 상대적으로 높으므로, split_transpose 제거에 따른 이득이 encoder 계열에 비해 다소 완만하게 나타난 것으로 해석할 수 있다.

한편, GEMM + Residual Add + LayerNorm + Bias Add Fusion은 현재 구현 및 최적화가 진행 중이며, 이에 대한 실험 설계 또한 요구사항에 포함된다. 이 feature는 QKV Fusion과 달리 단순 tensor transformation 제거에 그치지 않고, GEMM epilogue에 element-wise 연산과 reduction 기반 normalization을 함께 포함해야 하므로 구현 난도가 높다. 따라서 실험에서는 단순 latency뿐 아니라, fused kernel 내부에서의 reduction efficiency, shared memory 활용도, synchronization overhead, 그리고 numerical stability까지 함께 검토해야 한다. 예상되는 결과는 attention output projection 이후 또는 MLP projection 이후에 연속적으로 호출되던 residual add, bias add, layer normalization kernel이 통합됨에 따라 kernel launch 수와 intermediate tensor write/read가 감소하고, 특히 edge GPU 환경에서 더 큰 latency 개선이 나타나는 것이다. 다만 layer normalization의 reduction 비용 자체가 완전히 제거되는 것은 아니므로, QKV Fusion과 같은 직접적인 구조적 이득보다는 memory traffic 감소와 launch overhead 완화에 기반한 점진적인 성능 향상 형태로 나타날 가능성이 높다.

B. Overall System Architecture

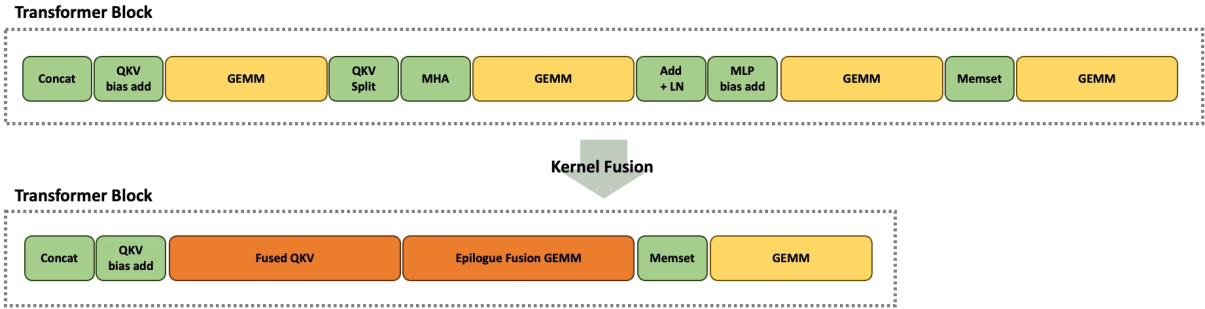


그림 6. 커널 퓨전을 적용한 이후의 전체 흐름

전체 시스템의 처리 흐름은 다음과 같다. 먼저, TensorRT 기반 멀티모달 모델을 baseline으로 설정하고, Nsight Systems를 이용하여 Transformer block 내부의 kernel 호출 순서와 실행 시간을 분석한다. 이후 Nsight Compute를 통해 각 kernel의 arithmetic intensity, DRAM access, memory throughput 특성을 분석하여 memory-bound kernel을 식별한다. 이러한 profiling 결과를 바탕으로, 본 연구에서는 Transformer block 내부에서 반복적으로 등장하는 비효율적인 연산 흐름을 대상으로 두 가지 핵심 fusion 기법, 즉 QKV Fusion과 GEMM Epilogue Fusion을 적용한다. 마지막으로, fused kernel이 적용된 시스템과 기존 TensorRT baseline을 비교하여 execution time, DRAM access, 그리고 latency 개선 효과를 정량적으로 평가한다.

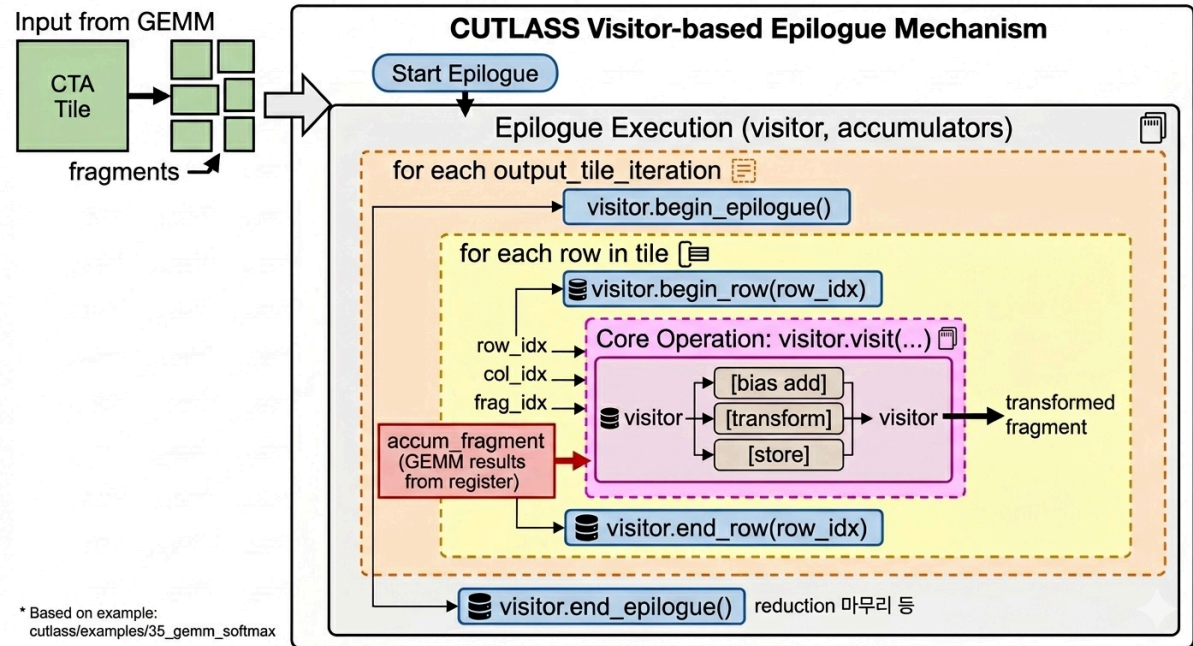


그림 7. CUTLASS 기반 Epilogue Fusion

시스템 아키텍처의 핵심은 CUTLASS 기반 GEMM kernel의 확장에 있다. 본 연구에서는 CUTLASS의 GEMM 구조를 기반으로 하되, GEMM 결과가 global memory로 기록되기 전에 epilogue 단계에서 추가 연산을 수행할 수 있도록 visitor 기반 epilogue processing 구조를 활용한다. CUTLASS의 epilogue는 accumulator에 저장된 GEMM 결과를 CTA tile 단위로 순회하면서 output fragment를 처리하는 구조를 가지며, 각 fragment에 대해 visitor가 순차적으로 호출되어 후처리를 수행한다. 구체적으로, epilogue 내부에서는 output tile iteration과 row loop를 따라 accumulator fragment가 visitor에 전달되며, visitor는 해당 fragment에 대해 bias addition, residual addition, tensor transform, reduction 기반 normalization, 그리고 최종 store 연산을 수행한다. 이와 같은 구조를 이용하면, 기존에는 별도의 CUDA kernel로 분리되어 있던 후처리 연산을 GEMM epilogue 내부에 통합할 수 있으므로, intermediate tensor를 global memory에 별도로 저장하지 않고도 연속적인 처리가 가능하다.

첨부한 kernel fusion 개념도는 이러한 시스템 재구성의 방향을 요약하여 보여준다. 기존 Transformer block은 `Concat`, `QKV bias add`, `GEMM`, `QKV split`, `MHA`, `GEMM`, `Add + LN`, `MLP bias add`, `GEMM`, `Memset`, `GEMM`과 같이 다수의 kernel이 순차적으로 호출되는 구조를 갖는다. 반면, 제안하는 시스템에서는 이 중 `QKV projection + split/transpose` 구간을 Fused QKV로 통합하고, `attention output projection + residual add + layer normalization + bias add` 구간을 Epilogue Fusion GEMM으로 통합한다. 그 결과, Transformer block 내부의 연산 흐름은 보다 긴 fused kernel 중심으로 재구성되며, 세분화된 kernel 호출 구조에서 발생하던 launch overhead와 memory traffic이 감소하게 된다. 즉, 본 시스템은 단순히 일부 연산을 묶는 수준이 아니라, Transformer block 내부의 데이터 이동 경로 자체를 축소하는 방향으로 전체 실행 구조를 재설계한 것이다.

제안하는 전체 시스템의 주요 설계 목표는 다음과 같이 정리할 수 있다. 첫째, memory-bound 특성이 강한 tensor transformation 및 post-processing kernel을 제거하거나 축소하여 DRAM access를 최소화한다. 둘째, GEMM 결과를 가능한 한 register 또는 on-chip memory 수준에서 후속 연산과 직접 연결함으로써 arithmetic intensity를 높인다. 셋째, 여러 개의 짧은 kernel을 소수의 fused kernel로 대체하여 kernel launch overhead를 줄인다. 넷째, 이러한 최적화가 특정 단일 모델에 국한되지 않도록, Transformer block 수준의 공통 연산 패턴을 기준으로 설계하여 CLIP, BERT, OPT, Llama 계열과 같은 다양한 encoder 및 decoder 구조에 적용 가능하도록 한다.

또한 본 시스템은 성능 개선뿐 아니라 실제 적용 가능성도 함께 고려한다. QKV Fusion은 attention preparation 단계에서 반복적으로 호출되는 split/transpose kernel을 제거함으로써 비교적 직접적인

latency 감소 효과를 기대할 수 있으며, sequence length가 증가할수록 그 효과가 더욱 크게 나타날 것으로 예상된다. GEMM Epilogue Fusion은 residual add, bias add, layer normalization과 같은 후처리 연산을 global memory round-trip 없이 수행하게 하므로, 특히 kernel launch overhead와 memory access 비용이 상대적으로 크게 작용하는 edge GPU 환경에서 유의미한 이점을 제공할 것으로 기대된다. 따라서 본 연구는 두 fusion 기법을 개별적으로 적용하는 데 그치지 않고, Transformer block 내부에서 연속적으로 결합하여 전체 block latency를 줄이는 방향으로 시스템을 구성한다.

종합하면, 본 연구의 전체 시스템은 profiling 기반 병목 식별, CUTLASS visitor 기반 fused epilogue 설계, QKV 및 post-processing kernel fusion, 그리고 baseline 대비 성능 검증의 네 단계로 구성된다. 이를 통해 기존 Transformer block의 fragmented execution structure를 fused execution structure로 변환하고, global memory access와 kernel launch overhead를 동시에 줄임으로써 멀티모달 모델 추론의 지연시간을 효과적으로 단축하는 것을 목표로 한다. 최종 산출물로는 멀티모달 모델의 Transformer block에 최적화된 kernel fusion 코드와, 이에 대한 설계·구현·실험 결과를 정리한 대한전자공학회 투고 논문을 제시한다.